PA1: Multiplier and Divider Design

Implement A 32-bit Unsigned Complete Multiplier

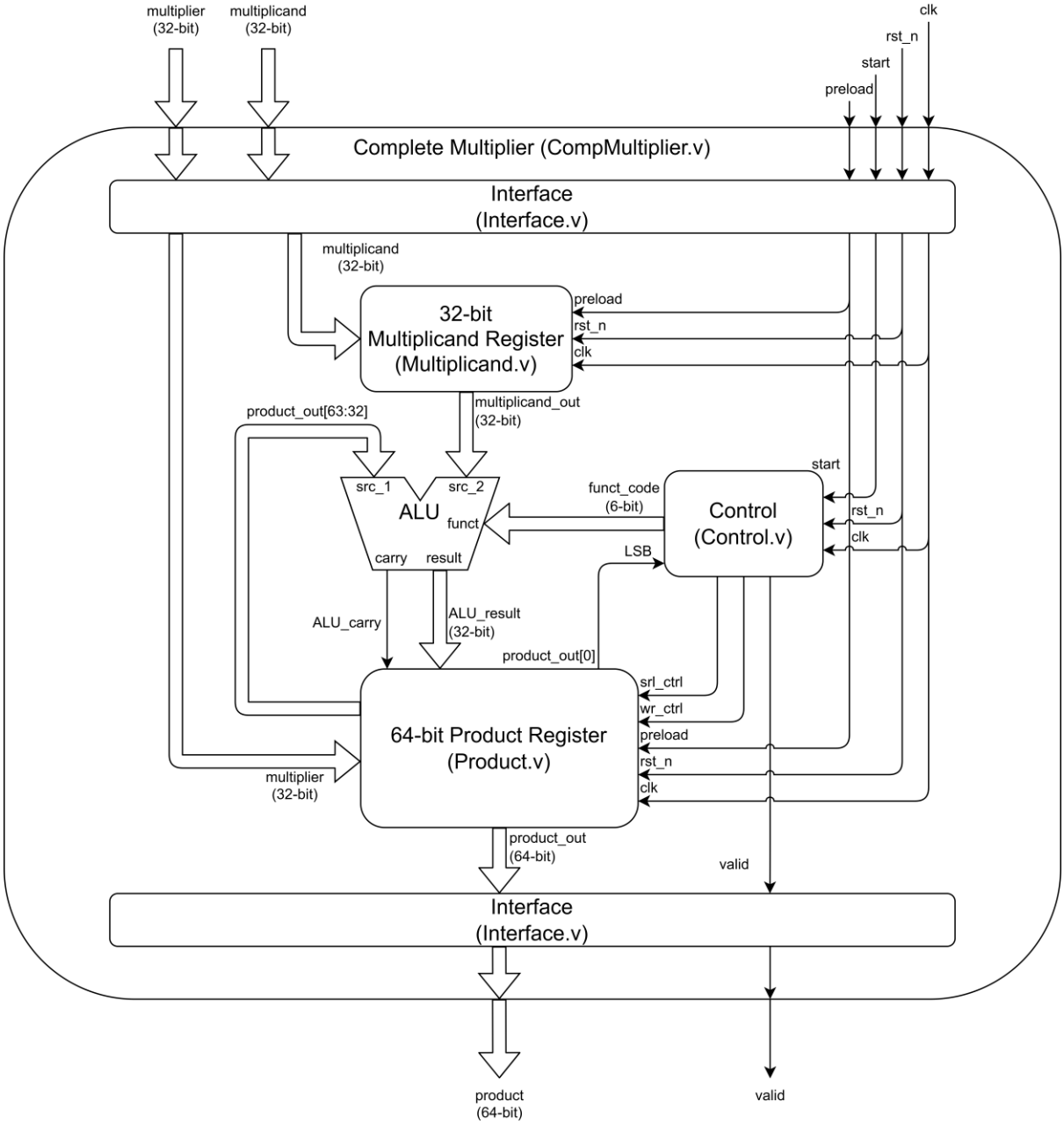

Fig. 1: 32-bit Unsigned Multiplier

Implement a 32-bit unsigned multiplier following the architecture shown in Fig. 1.

Design Constraints:

1. Do not use the Verilog multiplication operator (*) or any multiplication-related IP/primitive blocks.
2. Implement the multiplier using an iterative shift-and-add algorithm (i.e., repeated add and shift operations across multiple cycles).

Note:

- The **Interface** module is a set of positive edge triggered registers used to synchronize asynchronous I/O signals.

Module Definition

The module name and I/O ports of the top-level module **CompMultiplier** must **follow I/O interface definitions**. The names of the modules inside the system can be defined by the designer, but the number of modules should be equal to Fig. 1. The data bus and control bus in the system can be added or deleted by the designer. Fig. 1 is one of the solutions for the designer's reference.

<u>I/O Interface</u>

```
module CompMultiplier (
        input          clk,
        input          rst_n,
        input          preload,
        input          start,
        input  [31:0]  multiplier,
        input  [31:0]  multiplicand,
        output [63:0]  product,
        output         valid
);
```

Signal Description

Signal Name	I/O Format	Signal Description
clk	1-bit Input	System clock. All internal operations are triggered on the rising edge of clock.
rst_n	1-bit Input	**Synchronous negative edge** trigger reset. When asserted, all registers will be reset.
preload	1-bit Input	Load input control signal. When asserted, the inputs will be loaded into corresponding registers before activating the circuit.
start	1-bit Input	Start signal. When asserted and the multiplier is idle, a new multiplication operation is initiated.
multiplier	32-bit Input	Multiplier operand of the multiplication operation.
multiplicand	32-bit Input	Multiplicand operand of the multiplication operation.
product	64-bit Output	Product result of the multiplication operation.
valid	1-bit Output	Output valid signal. Asserted for one cycle when the multiplication result (product) is ready.

Operation Process

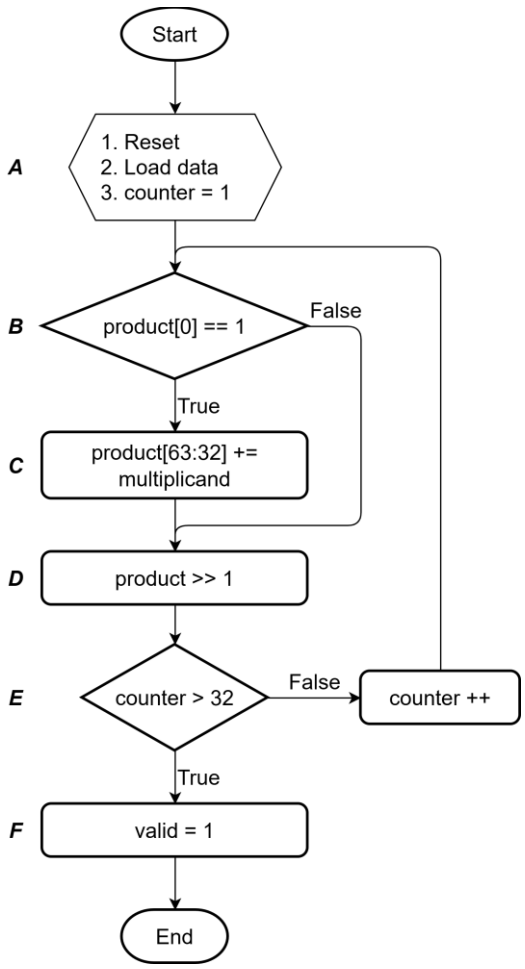

Fig. 2: Multiplication Flow Chart

Flowchart Description

Step A: The **rst_n** signal initializes the system. The **preload** signal triggers the reading of the **multiplicand** and **multiplier** and **start** signal starts the operation.

Step B: Check if the LSB of the **multiplier** is 1.

Step C: Accumulate the left half of the result register with the **multiplicand (**with ALU carry flag).

Step D: Shift the result register to the right by 1-bit.

Step E: If the counter is less than 32, increase the value of the counter by 1.

Step F: Set the **valid** signal as 1 to complete the multiplication.

Example (4-bit multiplication operation)

Counter.Step	**product**	**multiplicand**
	0000_0000	0000
0.A	0000_0011	0010
1.C	0010_0011	0010
1.D	0001_0001	0010
2.C	0011_0001	0010
2.D	0001_1000	0010
3.B	0001_1000	0010
3.D	0000_1100	0010
4.B	0000_1100	0010
4.D	0000_0110	0010
4.F	0000_0110	0010

Implement A 32-bit Unsigned Complete Divider

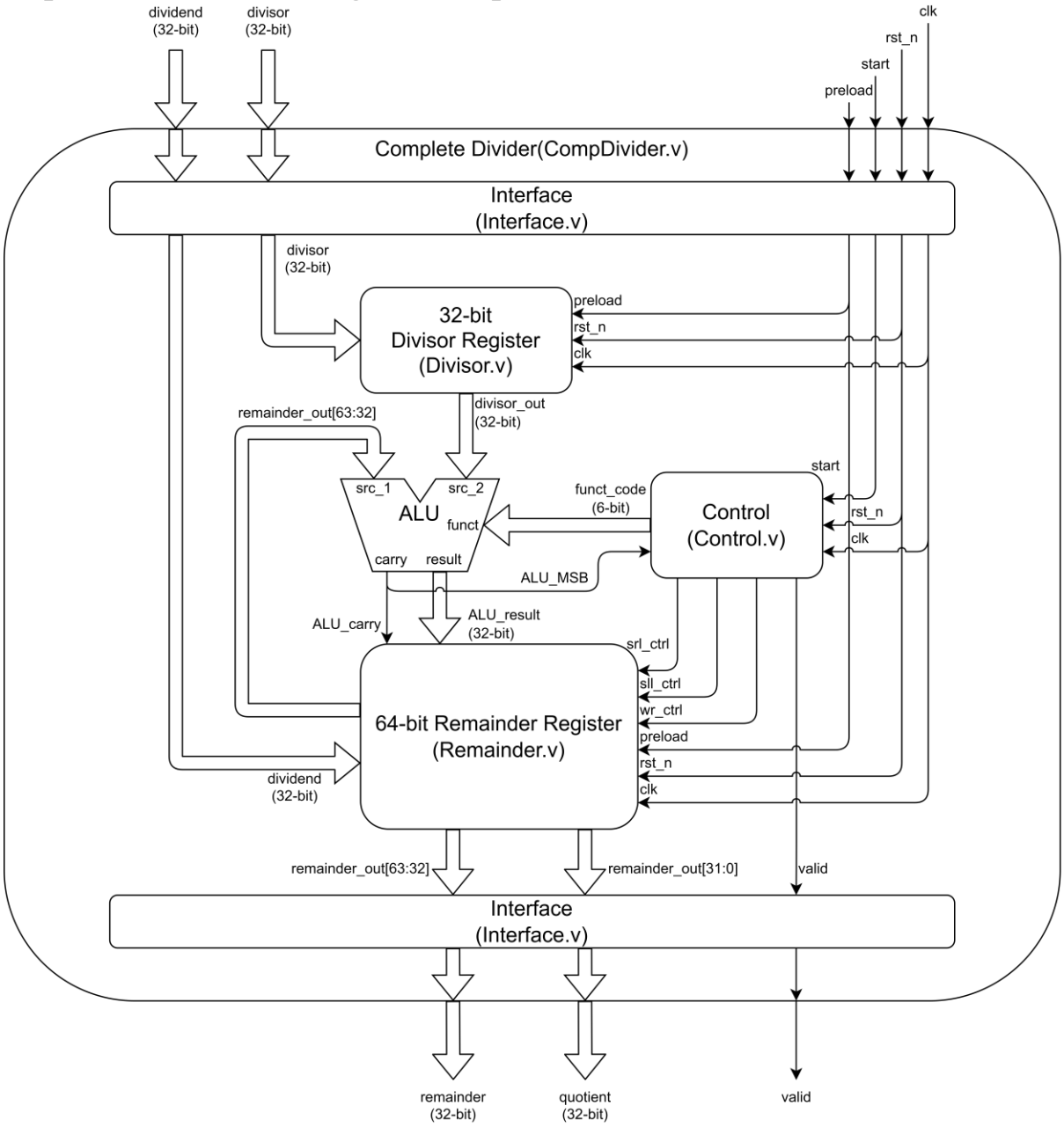

Fig. 3: 32-bit Unsigned Divider

Implement a 32-bit unsigned divider as shown in Fig. 3.

The design rules are the same as the multiplier, please refer to the content of the lecture notes for the operation process.

Design Constraints:

1. Do not use the Verilog division operator (e.g., A/B). Please use iterative shift-and-subtract algorithm to perform division.
2. Division by zero is **out of scope**.
3. Test vectors that produce a **zero quotient** are not included due to limitations of the optimized division algorithm used in this project.

I/O Interface

```
module CompDivider (
        input          clk,
        input          rst_n,
        input          preload,
        input          start,
        input   [31:0] divisor,
        input   [31:0] divident,
        output  [31:0] remainder,
        output  [31:0] quotient,
        output         valid
);
```

Signal Description

Signal symbol	Signal brief description
clk	1-bit Input clock signal
rst_n	1-bit Input synchronous negative edge reset signal
preload	1-bit Input load data control signal
start	1-bit Input start control signal
divisor	32-bit Input divisor data
dividend	32-bit Input dividend data
remainder	32-bit Output remainder result
quotient	32-bit Output quotient result
valid	1-bit Output valid control signal

Implement A 32-bit Complete Booth's Multiplier

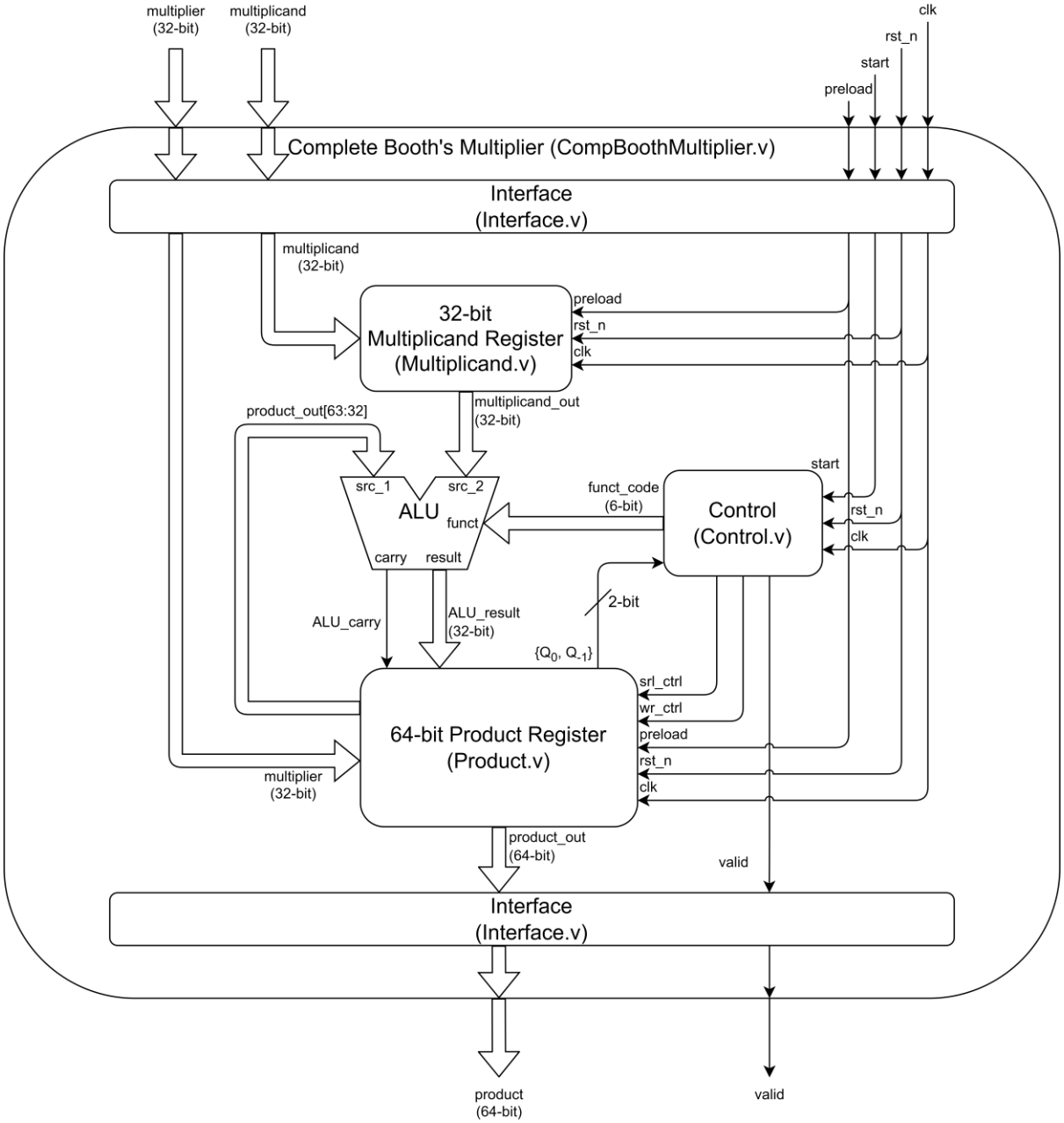

Fig. 4: 32-bit Booth's Multiplier

Implement a 32-bit booth's (signed) multiplier as shown in Fig. 4.

The design rules are the same and the architecture, design constraints are similar as the multiplier, please refer to the content of the lecture notes for the operation process.

<u>I/O Interface</u>

```
module CompBoothMultiplier (
        input          clk,
        input          rst_n,
        input          preload,
        input          start,
        input  [31:0]  multiplier,
        input  [31:0]  multiplicand,
        output [63:0]  product,
        output         valid
);
```

Signal Description

Signal symbol	Signal brief description
clk	1-bit Input clock signal
rst_n	1-bit Input synchronous negative edge reset signal
preload	1-bit Input load data control signal
start	1-bit Input start control signal
multiplier	32-bit Input multiplier data
multiplicand	32-bit Input multiplicand data
product	64-bit Output product result
valid	1-bit Output valid control signal

Design constraints for the above circuits:

1. The use of any **Look-Up-Table realization** is strictly **prohibited** in this project assignment.
2. Designers must strictly follow the specified **naming conventions** of all design **modules** and corresponding **I/O interfaces**.
3. The architecture design at the RTL level should be logically structured and reasonable.

Failure to meet the above requirements may result in point deductions.

Signal Flow of Simulation Testbench

Using **tb_CompMultiplier** as example, the testbench applies an active-low reset (**rst_n**) for a predefined duration before the first multiplication. After **rst_n** is deasserted, the testbench reads test vectors from **CompMultiplier_in.dat**, where each vector contains **multiplier**, **multiplicand** and the **golden product (product_golden)**.

For each test vector, the stimulus sequence is as follows (all signal updates occur on the **negative edge of the clock**):

1. Apply inputs: Drive **multiplier** and **multiplicand**, and assert **preload** to indicate the inputs should be captured by the interface registers.
2. Deassert preload: On the next negative edge, deassert **multiplier**, **multiplicand**, and **preload**.
3. Start operation: Assert **start** for one clock cycle to begin the iterative multiplication.
4. Wait for completion: The testbench waits until **valid** is asserted by your design.
5. Check result: When **valid** is high, the testbench samples **product** on the negative clock edge, compares it with **product_golden**, and writes the observed output to **CompMultiplier_out.dat**.
6. Next vector: After two additional clock cycles, the testbench proceeds to the next test vector and repeats the sequence.

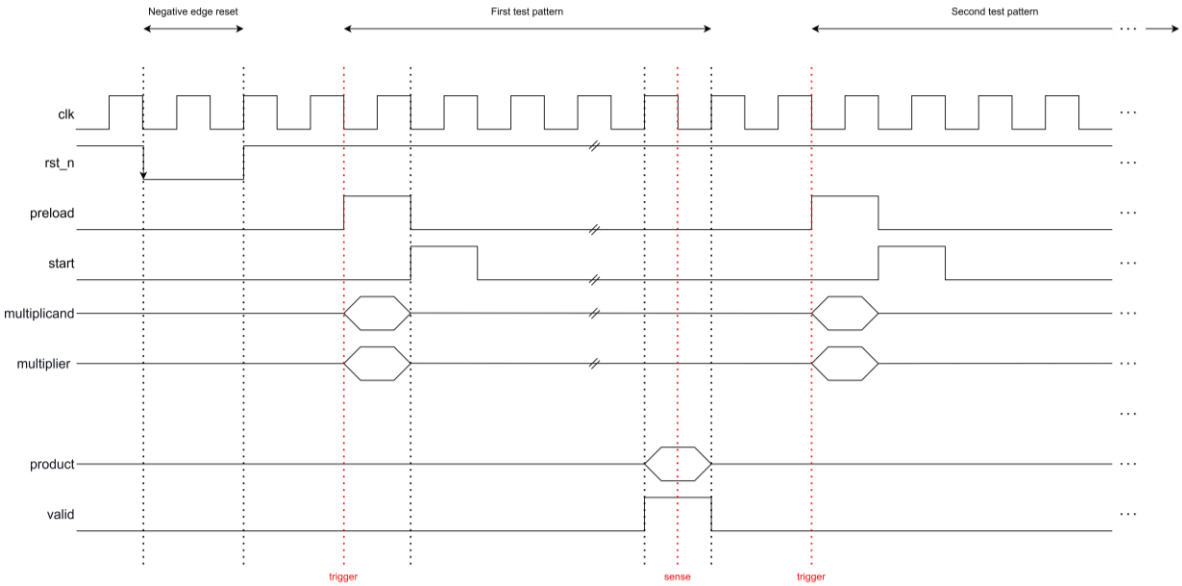

Fig. 5: Waveform Example

Note:

1. The testbench continuously waits for **valid** after **start** is asserted. If your design never asserts **valid**, the simulation will not terminate. Your design must guarantee that **valid** is asserted for every issued **start**.
2. Only the top-level module testbench is provided. You are responsible for verifying internal submodules during debugging. TAs will grade only the top-level design using the provided testbench. You may modify the testbench for debugging, but your final submission must pass the original testbench (in Fig. 8).
3. Test pattern files are written in hexadecimal.
   i. In **CompMultiplier_in.dat** and **CompBoothMultiplier_in.dat**: Each line contains three values separated by an underscore (_):

32-bit multiplier _ 32-bit multiplicand _ 64-bit golden product

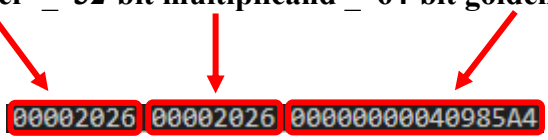

Fig. 6: Data Format in **CompMultiplier_in.dat**

   ii. In **CompDivider_in.dat**: Each line contains four values separated by an underscore (_):

32-bit dividend _ 32-bit divisor _ 32-bit golden remainder _ 32-bit golden quotient

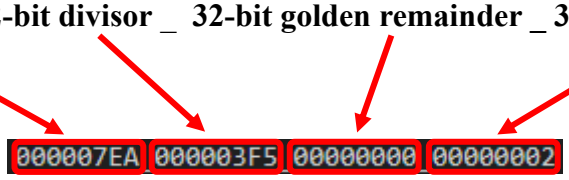

Fig. 7: Data Format in **CompDivider_in.dat**

4. You may add additional test vectors for self-testing.
5. Make sure output files (**CompMultiplier_out.dat, CompDivider_out.dat, CompBoothMultiplier_out.dat**) are generated and contain the expected results.

Fig. 8: Simulation Results in Vivado Tcl Console

Grading:

a. (35 pts) Report
b. (30 pts) Functionality of unsigned multiplier
   - In behavioral simulation
   - In post-synthesis timing simulation
c. (30 pts) Functionality of unsigned divider
   - In behavioral simulation
   - In post-synthesis timing simulation
d. (5 pts) Functionality of Booth's multiplier
   - In behavioral simulation
   - In post-synthesis timing simulation
e. **Follow the below specified naming rules and file format requirements.** Points may be deducted if the submission does not comply with the required rules and formats.
f. **No plagiarism**.

Note: We will test your modules with **another** generated **test patterns**.

Submission

Files required for PA1:

- BYYYDDXXX.zip

```
│
├─── CompMultiplier/
│    └── RTL/            # Verilog source files
│        ├── CompMultiplier.v
│        ├── Multiplicand.v
│        ├── ALU.v
│        ├── Product.v
│        ├── Interface.v
│        └── Control.v
│    └── SYN/            # Synthesized circuit file
│        └── BYYYDDXXX.dcp
│    └── XDC/            # Xilinx constraint file
│        └── BYYYDDXXX.xdc
├─── CompDivider/
│    └── RTL/            # Verilog source files
│        ├── CompDivider.v
│        ├── Divisor.v
│        ├── ALU.v
│        ├── Remainder.v
│        ├── Interface.v
│        └── Control.v
│    └── SYN/            # Synthesized circuit file
│        └── BYYYDDXXX.dcp
│    └── XDC/            # Xilinx constraint file
│        └── BYYYDDXXX.xdc
├─── CompBoothMultiplier/
│    └── RTL/            # Verilog source files
│        ├── CompBoothMultiplier.v
│        ├── Multiplicand.v
│        ├── ALU.v
│        ├── Product.v
│        ├── Interface.v
│        └── Control.v
│    └── SYN/            # Synthesized circuit file
│        └── BYYYDDXXX.dcp
│    └── XDC/            # Xilinx constraint file
│        └── BYYYDDXXX.xdc
│
└─── PDF/                # Report file
     └── BYYYDDXXX.pdf
```

- Note: Please make sure all your program files and PDF files are directly in the zipped file, not all wrapped in one folder.

Report structure:

1. Cover
2. Design Process
   i. **Describe** how you implement each module.
   ii. **Depict** your hardware architectures.
3. Behavioral Simulation
   i. **Describe** the test results (**including hexadecimal waveforms, Vivado tcl console**) of your modules in behavioral simulations.
4. Synthesis Result
   i. **Describe** the synthesis results (**including hardware utilization, timing**) of your modules.
5. Post-Synthesis Timing Simulation
   i. **Describe** the test results (**including hexadecimal waveforms, Vivado tcl console**) of your modules in post-synthesis timing simulations.
6. Conclusion
   i. Conclusion and insight in this project assignment.

Report Page Limitation:

The report should be within 15 pages (excluding the cover page).

Deadline: 2026-04-08 12:00 on Moodle

Appendix: **Cover Example**

Computer Organization, Spring 2026

Project Assignment 1:

Multiplier and Divider Design

Lecturer: 陳雅淑教授

Department: Electrical Engineering

Student ID: B11207XXX

Name: 林 X X